



École nationale supérieure de l'électronique et de ses
applications

Master 2 : **Ingénierie des Systèmes Complexes**
parcours Signaux, Information

**Prédiction intelligente de la charge et allocation
proactive des ressources dans les environnements
fog-cloud**

par :
EL-KHABLI Omar

Mme. SONIA YASSA
Mme. SOUAÏBOU HAWAOU KAYA

Encadrante
Co-encadrante

Année Universitaire : 2025–2026

TABLE DES MATIÈRES

1	Contexte général et positionnement du travail	2
1.1	Introduction	2
1.1.1	Context and Issues	2
1.1.2	Goals and Contributions	3
1.1.3	Paper organisation	4
1.2	Related Work	4
1.2.1	Allocation de ressources et équilibrage de charge	4
1.2.2	Ordonnancement intelligent des tâches	5
1.2.3	Prédiction de charge et auto-scaling proactif	5
1.2.4	Synthèse	6
1.3	System Model and Problem Formulation	7
1.3.1	Task (Application) Model	8
1.3.2	Nodes Model (Fog–Cloud Infrastructure)	8
1.3.3	QoS Metrics	8
1.3.4	Decision Variables and Constraints	9
1.3.5	Problem Formulation (Proactive Multi-objective Optimization)	10
1.4	Proposed Approach : Prédiction de charge et allocation proactive Fog–Cloud	11
1.4.1	Vue d’ensemble de l’approche	11
1.4.2	Architecture proposée	11
1.4.3	Module 1 — Prédiction de charge multi-horizon basée sur LSTM	13
1.4.4	Module 2 — Planning proactif : <i>scaling</i> et <i>offloading</i> guidés par la prédiction	15
1.4.5	Module 3 — Ordonnancement et allocation proactifs Fog–Cloud	18
1.4.6	Environnement de simulation et implémentation	20

CHAPITRE 1

CONTEXTE GÉNÉRAL ET POSITIONNEMENT DU TRAVAIL

1.1 Introduction

1.1.1 Context and Issues

L'Internet des Objets (IoT) joue aujourd'hui un rôle central dans l'industrie 4.0 et, en particulier, dans le secteur agro-alimentaire. Des dispositifs hétérogènes tels que les capteurs environnementaux, les drones, les stations météorologiques et les satellites génèrent en continu d'importants volumes de données exploitées pour des applications critiques, notamment la surveillance des cultures, l'irrigation intelligente, la détection précoce des maladies et l'optimisation des chaînes de production et de logistique. Ces applications imposent des contraintes strictes de latence, de fiabilité et de qualité de service (QoS), nécessitant un traitement en temps réel ou quasi temps réel des flux de données [7].

Le *cloud computing* offre des capacités élevées de calcul et de stockage ainsi qu'une forte élasticité, ce qui en fait une solution naturelle pour le traitement des données IoT à grande échelle. Toutefois, son architecture centralisée engendre des limitations importantes, notamment une latence accrue, une forte dépendance au réseau et des risques de congestion, qui peuvent dégrader significativement les performances des applications sensibles au délai. Ces limitations sont particulièrement critiques dans les systèmes agro-alimentaires, où des décisions tardives peuvent entraîner des pertes économiques ou environnementales.

Afin de répondre à ces contraintes, le paradigme du *fog computing* a été introduit pour rapprocher les ressources de calcul des sources de données. Le fog computing permet le déploiement de capacités de calcul, de stockage et de communication à proximité des dispositifs IoT, réduisant ainsi la latence et améliorant la réactivité des applications. L'intégration conjointe du fog et du cloud conduit aux architectures *fog-cloud*, dans lesquelles les traitements peuvent être répartis dynamiquement entre des

nœuds fog aux ressources limitées et des centres de données cloud puissants mais distants.

Cependant, la gestion des ressources dans les environnements *fog-cloud* demeure un défi majeur. Les nœuds fog sont contraints en termes de calcul, de stockage et d'énergie, et sont fortement distribués géographiquement. Par ailleurs, les charges de travail agro-alimentaires présentent une forte variabilité multi-échelles (journalière, saisonnière et annuelle), ainsi que des pics imprévisibles liés aux conditions climatiques, aux crises phytosanitaires ou aux perturbations logistiques. Une allocation inadéquate des ressources peut alors conduire soit à un sous-provisionnement dégradant la QoS et entraînant des violations de SLA, soit à un sur-provisionnement augmentant inutilement les coûts énergétiques et opérationnels [1].

Les solutions existantes de gestion des ressources dans les systèmes *fog-cloud* reposent majoritairement sur des mécanismes réactifs, déclenchés après le dépassement de seuils prédéfinis (CPU, mémoire ou latence). Bien que simples à implémenter, ces stratégies montrent rapidement leurs limites face à des charges fortement dynamiques et imprévisibles. Elles ne permettent pas d'anticiper les variations futures de la charge ni d'optimiser globalement les compromis entre latence, coût et utilisation des ressources.

Ces constats mettent en évidence la nécessité de solutions intelligentes et proactives, capables d'anticiper l'évolution de la charge et d'adapter à l'avance les décisions d'allocation. L'intégration de mécanismes de prédiction de charge basés sur l'intelligence artificielle, couplés à des stratégies d'allocation proactive des ressources, apparaît ainsi comme une approche prometteuse pour améliorer l'efficacité des environnements *fog-cloud*, garantir une QoS élevée et maîtriser les coûts dans les systèmes agro-alimentaires.

Problem Statement Dans ce contexte, la problématique centrale de ce travail peut être formulée comme suit :

Comment développer un système de prédiction intelligente capable d'anticiper les besoins en ressources computationnelles et de proposer des stratégies d'allocation proactive optimisant simultanément la latence et le coût dans un environnement agro-alimentaire fog-cloud ?

1.1.2 Goals and Contributions

Ce travail vise à proposer une approche intelligente pour la gestion proactive des ressources dans les environnements *fog-cloud*, en ciblant l'optimisation conjointe des métriques clés de Qualité de Service (QoS), notamment la latence, le coût opérationnel et l'utilisation efficace des ressources. L'objectif principal est de dépasser les stratégies réactives traditionnelles en anticipant l'évolution de la charge computationnelle des applications agro-alimentaires, caractérisées par une forte variabilité temporelle et des événements imprévisibles liés aux conditions climatiques et aux cycles agricoles.

Pour atteindre cet objectif, ce travail se concentre sur le développement d'un cadre

de prédiction intelligente de la charge, capable de capturer les variations multi-échelles (journalières, saisonnières et annuelles), et sur son intégration dans un mécanisme d'allocation proactive des ressources à travers le continuum *fog-cloud*. Cette allocation vise à déterminer, de manière anticipative, le dimensionnement, le placement et l'offloading des tâches entre les nœuds *fog* et le *cloud*, afin de garantir des performances stables tout en maîtrisant les coûts et la consommation énergétique.

Les principales contributions de ce travail peuvent être résumées comme suit. Premièrement, nous proposons un cadre conceptuel unifié combinant prédiction de charge et allocation proactive des ressources, spécifiquement adapté aux environnements agro-alimentaires *fog-cloud*. Deuxièmement, nous formulons le problème d'allocation des ressources comme une optimisation multi-objectifs prenant en compte simultanément la latence, le coût, l'énergie et les contraintes de QoS. Troisièmement, nous introduisons un mécanisme de décision hybride, associant des méthodes explicables de décision multi-critères et des techniques d'apprentissage par renforcement profond, afin de concilier robustesse, adaptabilité et faisabilité de déploiement. Enfin, un protocole expérimental reproductible est défini pour évaluer l'approche proposée et la comparer aux solutions de l'état de l'art en matière d'ordonnancement, d'auto-scaling et d'allocation de ressources dans les environnements *fog-cloud*.

1.1.3 Paper organisation

1.2 Related Work

La gestion des ressources dans les environnements *fog-cloud* a fait l'objet de nombreux travaux au cours des dernières années, portée par l'essor des applications IoT sensibles à la latence, au coût et à la consommation énergétique. Les approches existantes peuvent être globalement regroupées en trois grandes catégories : (i) l'allocation de ressources et l'équilibrage de charge, (ii) l'ordonnancement intelligent des tâches, et (iii) la prédiction de charge et l'auto-scaling proactif.

1.2.1 Allocation de ressources et équilibrage de charge

Plusieurs travaux se concentrent sur l'optimisation de l'allocation des ressources afin d'améliorer la Qualité de Service (QoS) dans les environnements distribués. **Mani** et **Meenakshisundaram** [8] proposent un cadre d'allocation de tâches et de placement de machines virtuelles dans un environnement Fog unique. Leur approche montre une réduction de la latence et du temps de réponse grâce à une meilleure exploitation des ressources disponibles. Toutefois, elle reste limitée à des scénarios relativement statiques et ne prend pas en compte la variabilité temporelle de la charge.

Dans une architecture intégrée Cloud-Fog, **Akintoye** et **Bagula** [2] formulent le problème d'allocation des tâches et de placement des machines virtuelles comme un problème d'optimisation. Leur solution hybride combine l'algorithme hongrois pour l'affectation des tâches et un algorithme génétique pour le placement des VMs. Les résultats expérimentaux montrent une réduction du coût d'allocation et une meilleure

utilisation des ressources. Néanmoins, cette approche demeure essentiellement réactive et ne s'appuie pas sur la prédiction de la charge future.

Dans une perspective multi-critères, **Gad-Elrab** et **Alsharkawy** [6] proposent une méthode d'équilibrage de charge dans les environnements Fog-Cloud basée sur la combinaison de FAHP et FTOPSIS pour classer les nœuds Fog selon plusieurs critères, tels que le temps de réponse, l'utilisation des ressources et la consommation énergétique. Cette approche améliore la stabilité du système et l'équilibrage global de la charge. Cependant, la décision d'allocation repose uniquement sur l'état instantané du système, sans mécanisme d'anticipation de l'évolution de la charge.

1.2.2 Ordonnancement intelligent des tâches

L'ordonnancement des tâches constitue un axe de recherche majeur en raison de l'hétérogénéité des ressources et des contraintes temporelles des applications IoT dans les environnements Fog-Cloud. Les approches basées sur l'apprentissage par renforcement profond ont récemment démontré un fort potentiel dans ce contexte.

Choppa et **Mangalampalli** [4] introduisent DRLMOTS, un ordonnanceur multi-objectif basé sur le Deep Q-Learning, capable de décider dynamiquement si une tâche doit être exécutée sur un nœud Fog ou déportée vers le Cloud. Les résultats expérimentaux, obtenus à partir de charges de travail réelles, montrent une réduction significative du *makespan* et de la consommation énergétique, ainsi qu'une amélioration de la tolérance aux pannes. Toutefois, cette approche reste centrée sur des décisions d'ordonnancement instantanées et ne prend pas en compte la prédiction de la charge à moyen ou long terme.

1.2.3 Prédiction de charge et auto-scaling proactif

La prédiction de charge est un élément clé pour la conception de mécanismes d'allocation proactive des ressources dans les environnements Cloud, Fog et Edge. Elle permet d'anticiper la demande future et de déclencher à l'avance des actions de provisionnement, réduisant ainsi les violations de QoS et le sur-provisionnement.

Jing Chen, **Yinglong Wang** [3] proposent une méthode proactive d'allocation de ressources dans le Cloud reposant sur une *prédiction adaptative des requêtes*. Leur approche combine un modèle de prédiction de charge (basé sur l'analyse historique des requêtes) avec une politique de provisionnement anticipatif. Les résultats montrent une réduction des coûts et du temps de réponse. Cependant, cette solution reste limitée aux architectures Cloud centralisées et ne considère pas les contraintes de latence des environnements Fog-Cloud.

Dans le contexte du fog computing, **Talaat** [10] introduit l'approche *EPRAM*, qui intègre un modèle de prédiction de charge basé sur des techniques d'apprentissage automatique, couplé à une stratégie d'allocation de ressources orientée QoS. Bien que les performances en termes de latence et de taux de satisfaction soient améliorées, l'approche est spécifique à un domaine applicatif (santé intelligente) et ne traite pas explicitement l'ordonnancement multi-objectifs ni le continuum Fog-Cloud.

Plus récemment, **Thang Le Duc, Chanh Nguyen** [5] proposent un cadre de prédiction de charge multi-horizon pour des applications Cloud–Edge à grande échelle. Leur approche repose sur des modèles de séries temporelles avancés (RNN/LSTM et variantes) afin de fournir des prévisions à court et moyen terme, destinées à guider les décisions de provisionnement. Toutefois, l’exploitation directe de ces prédictions dans un algorithme d’allocation ou d’ordonnancement Fog–Cloud reste peu explorée.

Enfin, **Pothu et Kailasam** [9] présentent un cadre d’auto-scaling proactif reposant sur un modèle hybride de prédiction de charge (*Tree-Enhanced Vector Machine*) et une phase de planification utilisant la métaheuristique *H-VWPO*. Cette approche améliore l’utilisation des ressources et le temps de réponse, mais elle est principalement orientée Cloud/IaaS et ne prend pas en compte les contraintes spécifiques des nœuds Fog.

1.2.4 Synthèse

En synthèse, bien que les travaux existants aient permis des avancées significatives en allocation de ressources, ordonnancement des tâches et auto-scaling, la majorité des approches demeurent soit réactives, soit partielles. Peu de solutions intègrent de manière conjointe la prédiction intelligente de la charge et l’allocation proactive multi-objectifs dans un environnement Fog–Cloud. Cette lacune est particulièrement marquée pour les applications agro-alimentaires, caractérisées par des charges à la fois cycliques et imprévisibles, ce qui motive la proposition d’une approche prédictive et proactive dans ce travail.

TABLE 1.1 – Comparaison des principales approches de gestion des ressources en environnements Fog-Cloud

Référence	Algo/Approche	Env -Outils	Objectifs / Limites	QoS
Mani & Meenakshi-sundaram (2020) [8]	ERAA	Fog (CloudSim)	Obj. Réduire latence/temps de réponse. Lim. Pas de prédiction, scénarios statiques.	L,T,C,E
Akintoye & Bagula (2019) [2]	GABVMP HABBP	Cloud-Fog (CloudSim + GUI Java)	Obj. Minimiser coût, améliorer utilisation. Lim. Réactif, pas proactif.	C,L,U
Gad-Elrab et al. (2024) [6]	AMCLBT FAHP+FTOPSIS	Fog-Cloud (non précisé)	Obj. Stabilité + équilibrage. Lim. État instantané, sans prédiction.	U,L,E,R
Choppara & Mangalam-palli (2024) [4]	DRLMOTS DQN	Fog-Cloud (SimPy)	Obj. Réduire makespan/énergie, tolérance aux fautes. Lim. Décisions instantanées.	MK,E,R
Talaat et al. (2022) [10]	EPRAM (RAM+EPM)	Fog-Cloud (iFog-Sim + Python)	Obj. Réduire temps réponse/SLA/énergie via prédiction. Lim. Dépend de la qualité des prédictions.	L,E,R,U
Chen et al. (2021) [3]	APMRT+RAMPVR + NSGA-II	Cloud (Alibaba trace; non précisé)	Obj. Réduire PM actifs, améliorer utilisation. Lim. Cloud centric, Fog non traité.	C,E,U,T
Le Duc et al. (2025) [5]	ARIMA/ES/LSTM + allocation	Cloud-Edge (Python)	Obj. Réduire sur/sous-provisionnement, améliorer QoS/SLA. Lim. Dépend du workload et du modèle.	L,U,R,C
Pothu & Kailasam (2024) [9]	H-VWPO+TreeSVM	Cloud (IaaS) (non précisé)	Obj. Utilisation ressources et temps de réponse. Lim. Peu adapté Fog/agro-alimentaire.	C,L,E

1.3 System Model and Problem Formulation

Nous considérons un environnement *fog-cloud* dédié à des applications IoT agro-alimentaires (surveillance de cultures, irrigation intelligente, détection précoce de maladies, traçabilité). Le système est organisé en couches : *Edge/IoT* (sources), *Fog* (ressources proches) et *Cloud* (ressources distantes mais abondantes)[7, 1]. L'objectif est de **prédire la charge** et de **décider proactivement** l'allocation des ressources et l'affectation des tâches.

1.3.1 Task (Application) Model

Soit $\mathcal{T}(t)$ l'ensemble des tâches arrivant durant une fenêtre de décision t . Chaque tâche $i \in \mathcal{T}(t)$ est caractérisée par les paramètres suivants :

- d_i : taille des données à transmettre et à traiter (en Mo) ;
- c_i : quantité de calcul requise, exprimée en millions d'instructions (MI) ;
- m_i : quantité de mémoire nécessaire à l'exécution de la tâche (en Mo) ;
- D_i : échéance temporelle (deadline) exprimée en secondes, définissant la contrainte de latence ;
- p_i : niveau de priorité de la tâche, reflétant son importance relative (par exemple, une alerte de détection de maladie est prioritaire par rapport à une analyse batch).

Ce type de modèle de tâches (taille de données, demande CPU/mémoire, deadline/priorité) est classiquement utilisé dans l'allocation et l'ordonnancement en environnements Fog-Cloud, notamment dans les approches d'ordonnancement DRL et les stratégies QoS [4, 2, 8].

1.3.2 Nodes Model (Fog-Cloud Infrastructure)

Soit $\mathcal{N} = \mathcal{N}_F \cup \mathcal{N}_C$ l'ensemble des nœuds, composé des nœuds fog $\mathcal{N}_F$ et des nœuds cloud $\mathcal{N}_C$. Chaque nœud $j \in \mathcal{N}$ est caractérisé par :

$$(F_j, M_j, \kappa_j, \pi_j)$$

où F_j est la capacité CPU (MIPS), M_j la mémoire disponible, κ_j un coefficient énergétique, et π_j un coût unitaire (souvent non nul au cloud). Cette distinction Fog (proche, limité, énergétiquement contraint) vs Cloud (puissant, distant, payant) correspond aux hypothèses adoptées dans les travaux d'allocation Fog-Cloud et d'équilibrage multi-critères [6, 2, 8].

La communication entre une source (ou passerelle) et un nœud j induit une latence réseau $L_{net}(i, j)$ et une bande passante disponible $B(i, j)$. Le temps de transfert est modélisé par :

$$T_{net}(i, j) = L_{net}(i, j) + \frac{d_i}{B(i, j)}$$

1.3.3 QoS Metrics

Temps d'exécution. Si la tâche i est exécutée sur le nœud j , son temps de calcul est approximé par :

$$T_{cpu}(i, j) = \frac{c_i}{f_j}$$

où f_j est la capacité de traitement effective (MIPS) disponible pour la tâche (selon la politique de partage)[8, 2].

Latence bout-en-bout. Le temps de complétion (latence) est :

$$T(i, j) = T_{net}(i, j) + T_{queue}(i, j) + T_{cpu}(i, j)$$

où $T_{queue}(i, j)$ représente l'attente (file) sur le nœud j (fonction de la charge)[8, 6].

Coût. Le coût monétaire (principalement cloud) peut être modélisé par :

$$C(i, j) = \pi_j \cdot c_i + \pi_j^{net} \cdot d_i$$

avec π_j^{net} un coût de transfert/usage réseau si nécessaire[2, 9].

Énergie. Une forme simple de consommation énergétique au fog :

$$E(j) = E_j^{idle} + \kappa_j \cdot u_j$$

où u_j est l'utilisation CPU (ou une charge normalisée) sur la fenêtre considérée[4, 6].

Violations SLA/Deadline. Une violation est comptée si $T(i, j) > D_i$. On définit [9, 8] :

$$V(i, j) = \mathbb{1}[T(i, j) > D_i].$$

1.3.4 Decision Variables and Constraints

À chaque fenêtre temporelle t , le modèle décide de l'affectation des tâches aux ressources *fog* et/ou *cloud*. Dans le cas d'une approche proactive, il détermine également le niveau de provisionnement (scaling) des ressources correspondantes.

Affectation des tâches. La variable binaire $x_{i,j}(t)$ indique si la tâche i est affectée au nœud j à l'instant t [4, 2] :

$$x_{i,j}(t) \in \{0, 1\}, \quad \sum_{j \in \mathcal{N}} x_{i,j}(t) = 1, \quad \forall i \in \mathcal{T}(t).$$

Provisionnement (scaling). On note $s_j(t)$ la capacité provisionnée (par exemple, le nombre de conteneurs ou de machines virtuelles) sur le nœud ou le cluster j . Dans un modèle simplifié, on impose [9] :

$$0 \leq s_j(t) \leq s_j^{\max}, \quad \forall j \in \mathcal{N}.$$

Contraintes de capacité. Les ressources de calcul et de mémoire disponibles sur chaque nœud doivent suffire à l'exécution des tâches affectées [8, 6] :

$$\sum_{i \in \mathcal{T}(t)} x_{i,j}(t) c_i \leq F_j(s_j(t)), \quad \sum_{i \in \mathcal{T}(t)} x_{i,j}(t) m_i \leq M_j(s_j(t)), \quad \forall j \in \mathcal{N}.$$

Contraintes de QoS. Pour les tâches critiques, une contrainte stricte de délai peut être imposée :

$$T(i, j) \leq D_i, \quad \forall i \in \mathcal{T}_{\text{crit}}(t).$$

Alternativement, les violations de QoS peuvent être intégrées sous forme de pénalités dans la fonction objectif, ce qui permet une modélisation plus flexible en situation de forte charge [2, 9].

1.3.5 Problem Formulation (Proactive Multi-objective Optimization)

Notre problème consiste à **anticiper** la charge via $\hat{w}(t + \tau)$ et à choisir $x(t) = \{x_{i,j}(t)\}$ et $s(t) = \{s_j(t)\}$ afin d'optimiser simultanément plusieurs objectifs.

Objectifs. Sur une fenêtre t , on définit [8, 2, 6, 4, 9] :

$$\text{Latence : } J_L(t) = \sum_{i \in \mathcal{T}(t)} \sum_{j \in \mathcal{N}} x_{i,j}(t) T(i, j),$$

$$\text{Coût : } J_C(t) = \sum_{i \in \mathcal{T}(t)} \sum_{j \in \mathcal{N}} x_{i,j}(t) C(i, j),$$

$$\text{Énergie : } J_E(t) = \sum_{j \in \mathcal{N}_F} E(j),$$

$$\text{Violations SLA : } J_V(t) = \sum_{i \in \mathcal{T}(t)} \sum_{j \in \mathcal{N}} x_{i,j}(t) V(i, j).$$

Optimisation multi-objectifs. Le problème peut être traité de deux manières :

1. **Somme pondérée** (pratique pour l'implémentation) :

$$\min_{x(t), s(t)} \alpha_L J_L(t) + \alpha_C J_C(t) + \alpha_E J_E(t) + \alpha_V J_V(t),$$

avec $\alpha_L, \alpha_C, \alpha_E, \alpha_V \geq 0$ réglant les compromis ;

2. **Pareto-optimalité** (analyse plus riche) : minimiser simultanément (J_L, J_C, J_E, J_V) sous contraintes.

Proactivité basée sur prédiction. Pour rendre l'allocation proactive, les décisions de scaling peuvent dépendre explicitement de la charge prédite :

$$s(t) = G(\hat{w}(t + \tau), w(t), \text{état réseau}),$$

où $G(\cdot)$ est une politique (heuristique multi-critères, optimisation, ou DRL) visant à provisionner *avant* la saturation. Les affectations $x(t)$ exploitent ensuite les capacités provisionnées pour minimiser latence/coût tout en limitant les violations SLA.

1.4 Proposed Approach : Prédiction de charge et allocation proactive Fog–Cloud

Cette section présente une approche s'appuyant sur des méthodes éprouvées et reconnues dans les travaux existants, visant à intégrer la **prédiction de charge** au sein d'un mécanisme d'**allocation proactive** des ressources dans un environnement Fog–Cloud dédié aux applications agro-alimentaires. L'objectif n'est pas de proposer un nouveau modèle de prédiction complexe, mais de **mettre en évidence l'apport concret de la prédiction de charge sur la prise de décision** en matière de *scaling*, d'*offloading* et d'ordonnancement des tâches.

L'approche s'inscrit dans une boucle de décision de type *Monitoring–Analysis–Planning–Execution (MAPE)*, couramment adoptée dans les mécanismes d'auto-scaling proactif [9], et repose sur des algorithmes validés et largement utilisés dans les environnements Fog–Cloud [4, 6].

1.4.1 Vue d'ensemble de l'approche

Le fonctionnement global repose sur une prise de décision par fenêtres temporelles discrètes t . À chaque fenêtre, les étapes suivantes sont exécutées :

1. **Monitoring** : collecte des métriques système (taux d'arrivée des tâches, utilisation CPU et mémoire, longueur des files d'attente, latence réseau) ainsi que des variables contextuelles (heure, jour, saison, conditions météorologiques) [1, 7].
2. **Prediction** : estimation de la charge future à plusieurs horizons $\tau \in \{5, 15, 30, 60\}$ minutes, afin d'anticiper les variations de demande [9].
3. **Planning proactif** : décision anticipée de mise à l'échelle des ressources (*scaling*) et de répartition Fog/Cloud (*offloading*), avant l'apparition de la congestion.
4. **Scheduling / Allocation** : affectation des tâches aux nœuds Fog ou Cloud en tenant compte des priorités et des contraintes de QoS, via des mécanismes inspirés de l'apprentissage par renforcement profond [4] et de la décision multi-critères [6].
5. **Feedback** : comparaison entre la charge prédite et la charge réelle, permettant d'ajuster les paramètres de décision et d'améliorer la robustesse du système.

1.4.2 Architecture proposée

Nous proposons une architecture proactive Figure 1.1 pour l'ordonnancement et l'allocation des ressources dans un environnement Fog–Cloud, visant à garantir la qualité de service (QoS) face à des charges de travail dynamiques et incertaines. L'architecture s'organise sous la forme d'un workflow décisionnel adaptatif en boucle fermée, inspiré du modèle *Monitoring–Analysis–Planning–Execution (MAPE)*, permettant une adaptation continue du système en fonction de l'état observé et des prévisions de charge.

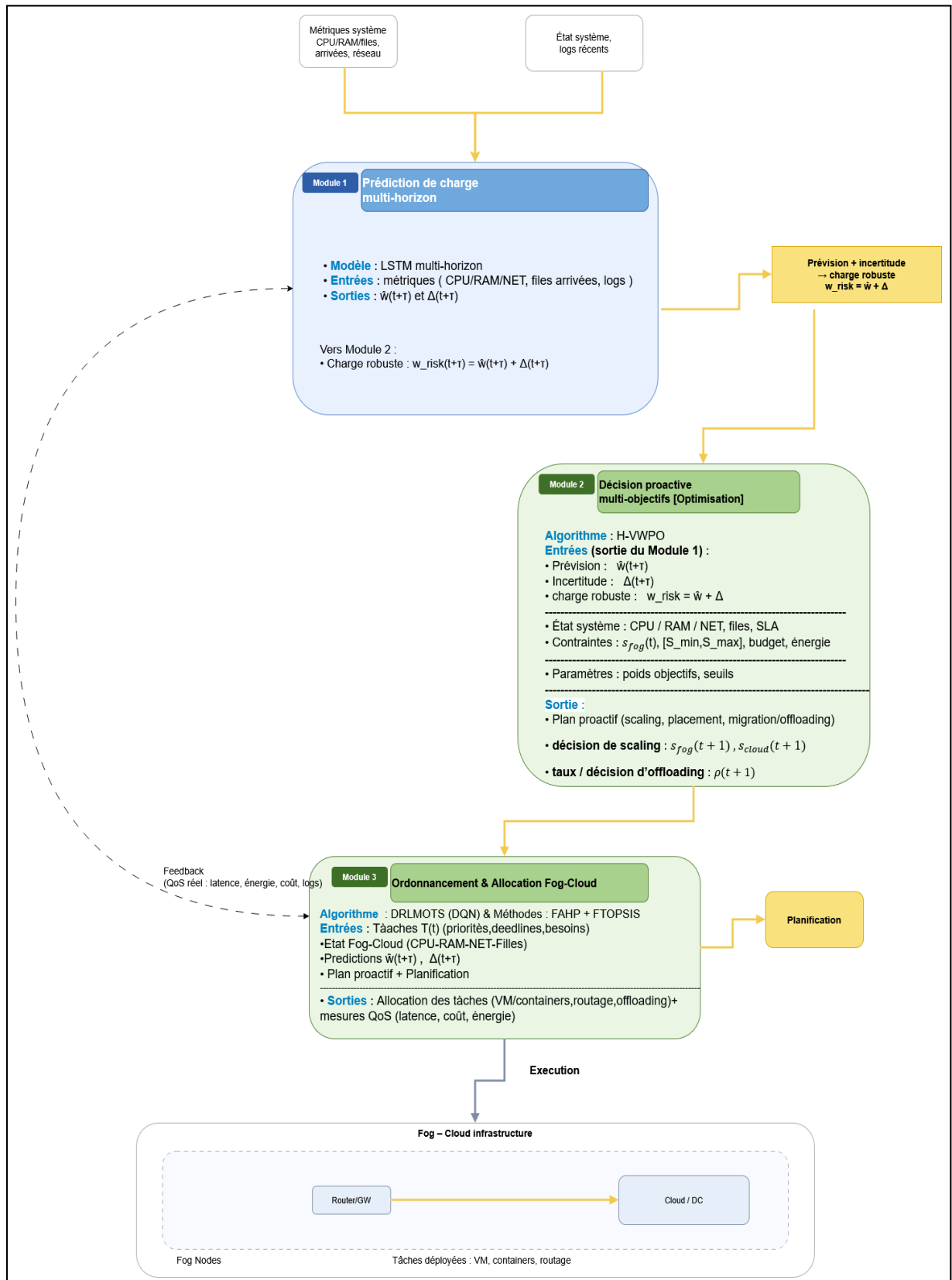


FIGURE 1.1 – Architecture proposée

Dans un premier temps, les métriques système (utilisation CPU et mémoire, état du réseau, longueurs des files d'attente et taux d'arrivée des tâches), ainsi que l'état courant de l'infrastructure et les logs récents, sont collectés afin de caractériser le comportement instantané du système. Ces informations constituent les données d'entrée du module de prédiction de charge.

Le **Module 1**_{1.4.3} est dédié à la prédiction multi-horizon de la charge à l'aide d'un modèle basé sur les *Long Short-Term Memory* (LSTM). À partir de l'historique des métriques système et de variables contextuelles, le modèle fournit, pour plusieurs horizons temporels, une estimation de la charge future accompagnée d'une mesure d'incertitude. Afin de limiter les risques liés aux erreurs de prédiction et aux fluctuations imprévues de la demande, une charge robuste est construite en combinant la prévision et sa marge d'incertitude. Cette charge sécurisée sert de base aux décisions proactives ultérieures.

Sur la base de cette charge robuste, le **Module 2**_{1.4.4} assure la phase de *planning* proactif multi-objectifs. Il détermine un plan de provisionnement et de répartition des ressources Fog–Cloud, incluant des décisions de mise à l'échelle (*scaling*) et d'offloading. Ces décisions visent à anticiper les situations de congestion côté Fog et à mobiliser le Cloud comme capacité élastique de secours, tout en respectant les contraintes de capacité, de coût, d'énergie et de SLA.

Le **Module 3**_{1.4.5} réalise ensuite l'ordonnancement et l'allocation proactifs des tâches à une granularité fine. En s'appuyant sur les décisions de scaling et d'offloading issues du **Module 2**_{1.4.4}, ainsi que sur les prédictions de charge du **Module 2**_{1.4.3}, ce module détermine pour chaque tâche si elle doit être exécutée localement au Fog ou déportée vers le Cloud. Lorsque l'exécution Fog est retenue, le module sélectionne le nœud Fog le plus approprié en tenant compte de critères de QoS tels que la latence, la charge des ressources et la consommation énergétique. Les décisions d'ordonnancement sont ensuite appliquées à l'infrastructure Fog–Cloud via le déploiement de conteneurs ou de machines virtuelles et la configuration du routage.

Enfin, les métriques de QoS observées après exécution (latence réelle, coût, énergie, violations SLA) sont collectées et renvoyées sous forme de feedback vers les modules amont. Ce retour d'information permet d'ajuster dynamiquement les paramètres de prédiction et de décision, renforçant ainsi la robustesse et l'efficacité globale du système.

1.4.3 Module 1 — Prédiction de charge multi-horizon basée sur LSTM

Pour la prédiction de charge, nous adoptons un modèle basé sur les *Long Short-Term Memory* (LSTM), en nous appuyant sur des travaux récents consacrés à la prédiction de charge dans les environnements Cloud–Edge. Bien que le cadre étudié combine plusieurs modèles (statistiques et apprentissage profond), nous retenons ici

uniquement le modèle LSTM afin de maintenir un bon compromis entre précision, complexité et facilité d'implémentation [5].

Ce choix s'explique par la capacité des LSTM à modéliser des dépendances temporelles à long terme et des dynamiques non linéaires, tout en restant robustes face à la non-stationnarité des séries temporelles. De plus, leur efficacité a été largement validée dans des scénarios IoT et Fog-Cloud caractérisés par une forte variabilité de charge [1, 7]. Dans notre cas d'étude agro-alimentaire, la présence de cycles journaliers, hebdomadaires et saisonniers renforce la pertinence de ce modèle.

Comme illustré à la Figure 1.2, le module de prédiction repose sur l'exploitation de l'historique des métriques système afin d'anticiper l'évolution de la charge à plusieurs horizons temporels.

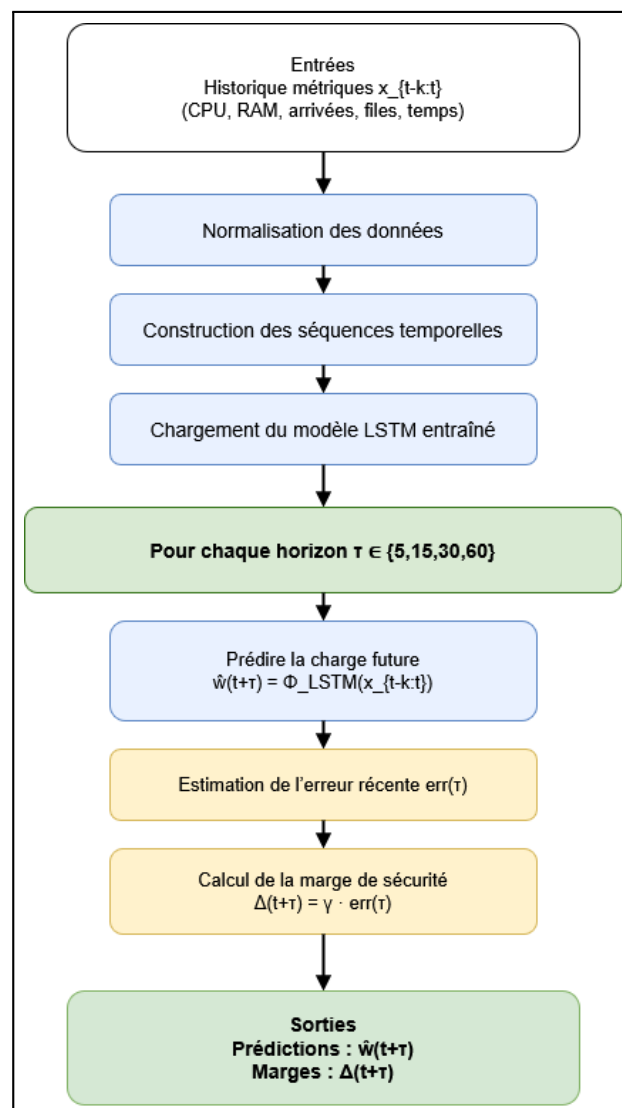


FIGURE 1.2 – Diagramme du module de prédiction de charge multi-horizon basé sur LSTM

À partir de l'historique des observations système $\mathbf{x}_{t-k:t} = \{\mathbf{x}_{t-k}, \dots, \mathbf{x}_t\}$, compre-

nant l'utilisation CPU et mémoire, les taux d'arrivée des tâches, les longueurs des files d'attente, ainsi que des variables exogènes (heure, jour, saison), le modèle LSTM apprend une fonction de prédiction de la forme :

$$\hat{w}(t + \tau) = \Phi_{\text{LSTM}}(\mathbf{x}_{t-k:t}), \quad \tau \in \{5, 15, 30, 60\}.$$

Pour chaque horizon τ , le modèle fournit une estimation ponctuelle de la charge $\hat{w}(t + \tau)$ ainsi qu'une marge d'incertitude $\Delta(t + \tau)$, construite à partir des erreurs de prédiction récentes. Cette marge permet d'intégrer explicitement l'incertitude associée aux prévisions et de produire une estimation de charge plus robuste, utilisée par les modules décisionnels en aval [5].

L'Algorithme 1 formalise le processus de prédiction de charge multi-horizon basé sur LSTM.

Algorithm 1 Prédiction de charge multi-horizon basée sur LSTM

Require: Historique des métriques $\mathbf{x}_{t-k:t}$, horizons $\mathcal{T} = \{5, 15, 30, 60\}$

Ensure: Prédictions $\hat{w}(t + \tau)$ et marges $\Delta(t + \tau)$

- 1: Normaliser les données d'entrée
 - 2: Construire les séquences temporelles à partir de $\mathbf{x}_{t-k:t}$
 - 3: Charger le modèle LSTM entraîné hors-ligne
 - 4: **for** chaque horizon $\tau \in \mathcal{T}$ **do**
 - 5: Prédire la charge future $\hat{w}(t + \tau)$ via le LSTM
 - 6: Estimer l'erreur récente $err(\tau)$
 - 7: Calculer la marge de sécurité $\Delta(t + \tau) = \gamma \cdot err(\tau)$
 - 8: **end for**
 - 9: **return** $\{\hat{w}(t + \tau), \Delta(t + \tau)\}_{\tau \in \mathcal{T}}$
-

1.4.4 Module 2 — Planning proactif : *scaling* et *offloading* guidés par la prédiction

Le module de *planning* proactif s'inscrit dans une boucle autonome de type *Monitoring–Analysis–Planning–Execution* (MAPE), couramment adoptée dans les mécanismes d'auto-scaling proactif fondés sur la prédiction. À chaque fenêtre temporelle, l'état du système est analysé à partir des métriques collectées, puis des décisions anticipées de mise à l'échelle des ressources sont planifiées et exécutées. Le retour d'exécution est ensuite exploité afin d'ajuster les décisions futures [9]. Dans notre contexte Fog–Cloud, cette approche vise principalement à prévenir la congestion côté Fog en mobilisant le Cloud comme capacité élastique de secours [1, 7].

Dans la phase de **Planning**, nous adoptons la méta-heuristique hybride **H-VWPO** (*Hybrid Vultures and Waterwheel Plant Optimization*), initialement proposée pour l'auto-scaling guidé par la prédiction de charge [9]. Cette approche combine *AVOA* (*African Vultures Optimization Algorithm*) afin de favoriser l'exploration globale de l'espace de recherche, et *WWPA* (*Waterwheel Plant Optimization Algorithm*) pour renforcer l'exploitation locale et éviter la stagnation dans des optima locaux. Ce choix est motivé par

l'adéquation directe de H-VWPO au problème étudié : l'algorithme opère à partir d'un état analysé et prédit du système et génère directement des décisions de provisionnement des ressources, sans nécessiter la conception d'une méthode d'optimisation ad hoc [9].

Dans [9], H-VWPO est évalué dans un contexte Cloud multi-couches. Dans notre cadre Fog–Cloud, l'algorithme est réutilisé sans modification structurelle, avec une adaptation sémantique des couches. Le **Fog** est considéré comme une couche de proximité, à faible latence mais à capacité limitée, destinée à l'exécution prioritaire des tâches sensibles à la latence, tandis que le **Cloud** est vu comme un réservoir de ressources élastiques activé lorsque la demande prédite dépasse la capacité Fog disponible, y compris après une opération de *scale-up*. Dans ce cas, les tâches non critiques sont *offloadées* vers le Cloud afin de préserver la QoS [1, 7]. En conséquence, H-VWPO est principalement responsable des décisions de *scaling*, tandis que la politique d'*offloading* est dérivée a posteriori du *gap* entre la charge prédite sécurisée et la capacité Fog.

Comme illustré à la Figure 1.3, le Module 2 exploite la charge prédite robuste issue du Module 1 afin de rechercher une configuration optimale de capacité Fog–Cloud . La phase d'analyse fournit des indicateurs d'état des ressources (*over-utilized*, *under-utilized*, *normal*), tandis que la phase de planning utilise H-VWPO pour minimiser les pénalités SLA et améliorer l'utilisation des ressources et le temps de réponse. La logique de décision finale suit le principe décrit dans [9] : un état majoritairement *over-utilized* conduit à un *scale-up*, un état *under-utilized* à un *scale-down*, sinon l'état est considéré comme *normal*.

L'Algorithme 2 formalise la stratégie de planification proactive Fog–Cloud reposant sur la méta-heuristique H-VWPO pour les décisions de *scaling* et d'*offloading*.

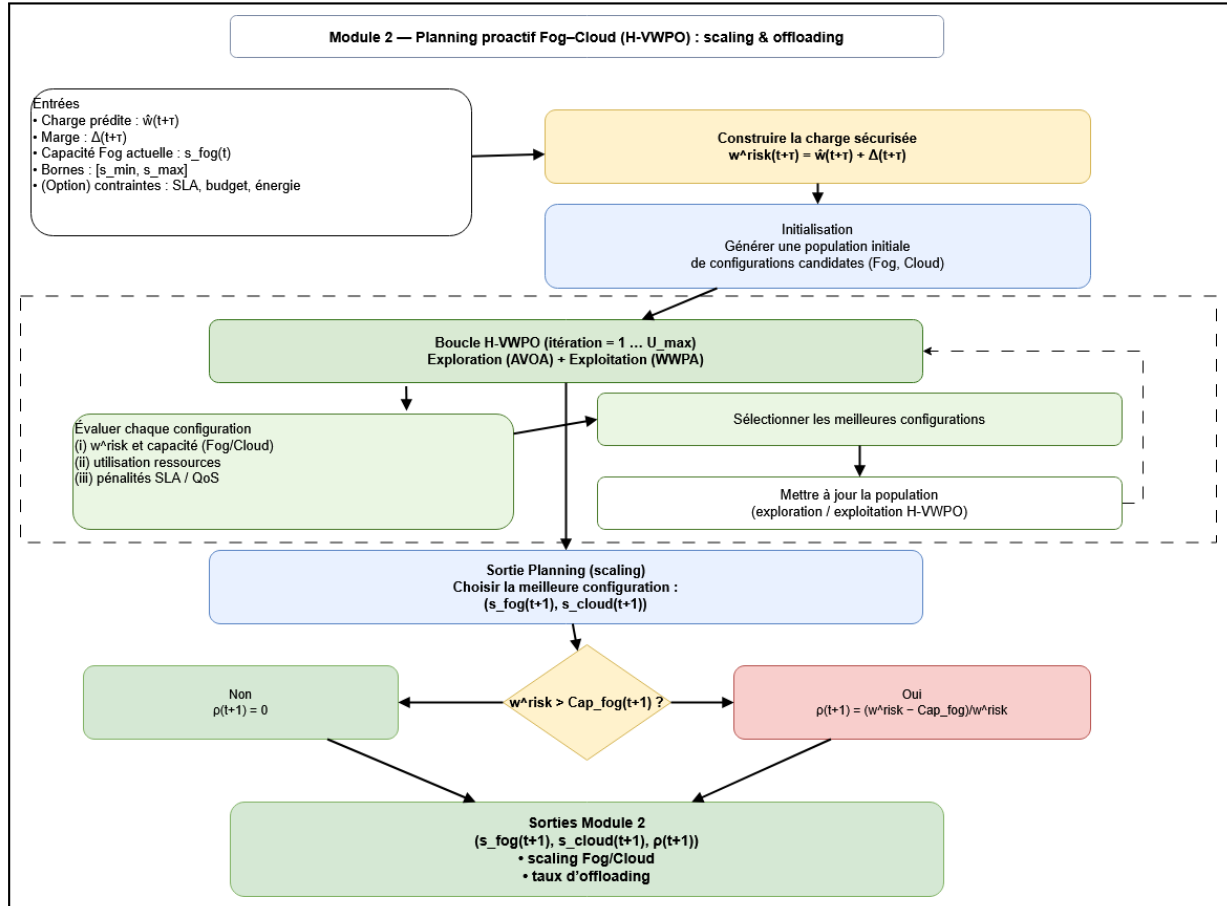


FIGURE 1.3 – Diagramme du Module 2 : planning proactif Fog-Cloud basé sur H-VWPO (scaling et offloading)

Algorithm 2 Planning proactif Fog-Cloud basé sur H-VWPO

Require: Charge prédite $\hat{w}(t + \tau)$, marge $\Delta(t + \tau)$

Require: Capacité Fog actuelle $s_{fog}(t)$, bornes $[s_{min}, s_{max}]$

Ensure: Décision de scaling ($s_{fog}(t+1), s_{cloud}(t+1)$) et offloading $\rho(t+1)$

- 1: Générer une population initiale de configurations candidates (Fog, Cloud)
- 2: **for** itération = 1 à U_{max} **do**
- 3: Évaluer chaque configuration selon :
- 4: (i) charge prédite sécurisée $w^{risk} = \hat{w} + \Delta$
- 5: (ii) utilisation des ressources
- 6: (iii) pénalités SLA
- 7: Sélectionner les meilleures configurations
- 8: Mettre à jour les configurations par exploration / exploitation (H-VWPO)
- 9: **end for**
- 10: Sélectionner la meilleure configuration ($s_{fog}(t+1), s_{cloud}(t+1)$)
- 11: **if** $w^{risk} > Cap_{fog}(t+1)$ **then**
- 12: $\rho(t+1) \leftarrow \frac{w^{risk} - Cap_{fog}}{w^{risk}}$
- 13: **else**
- 14: $\rho(t+1) \leftarrow 0$
- 15: **end if**
- 16: **return** ($s_{fog}(t+1), s_{cloud}(t+1), \rho(t+1)$)

1.4.5 Module 3 — Ordonnancement et allocation proactifs Fog–Cloud

Le Module 3 assure l'ordonnancement et l'allocation proactifs des tâches dans l'environnement Fog–Cloud. Il s'appuie sur les décisions de *scaling* et d'*offloading* produites par le Module 2, ainsi que sur les prévisions de charge issues du Module 1, afin d'orienter l'exécution des tâches à une granularité fine. Pour chaque tâche, le module détermine si elle doit être exécutée localement au niveau Fog ou déportée vers le Cloud et, lorsque l'exécution Fog est retenue, sélectionne le nœud Fog le plus approprié en fonction des critères de QoS.

Comme illustré à la Figure 1.4, l'ordonnancement est réalisé selon un processus décisionnel hiérarchique en deux niveaux complémentaires.

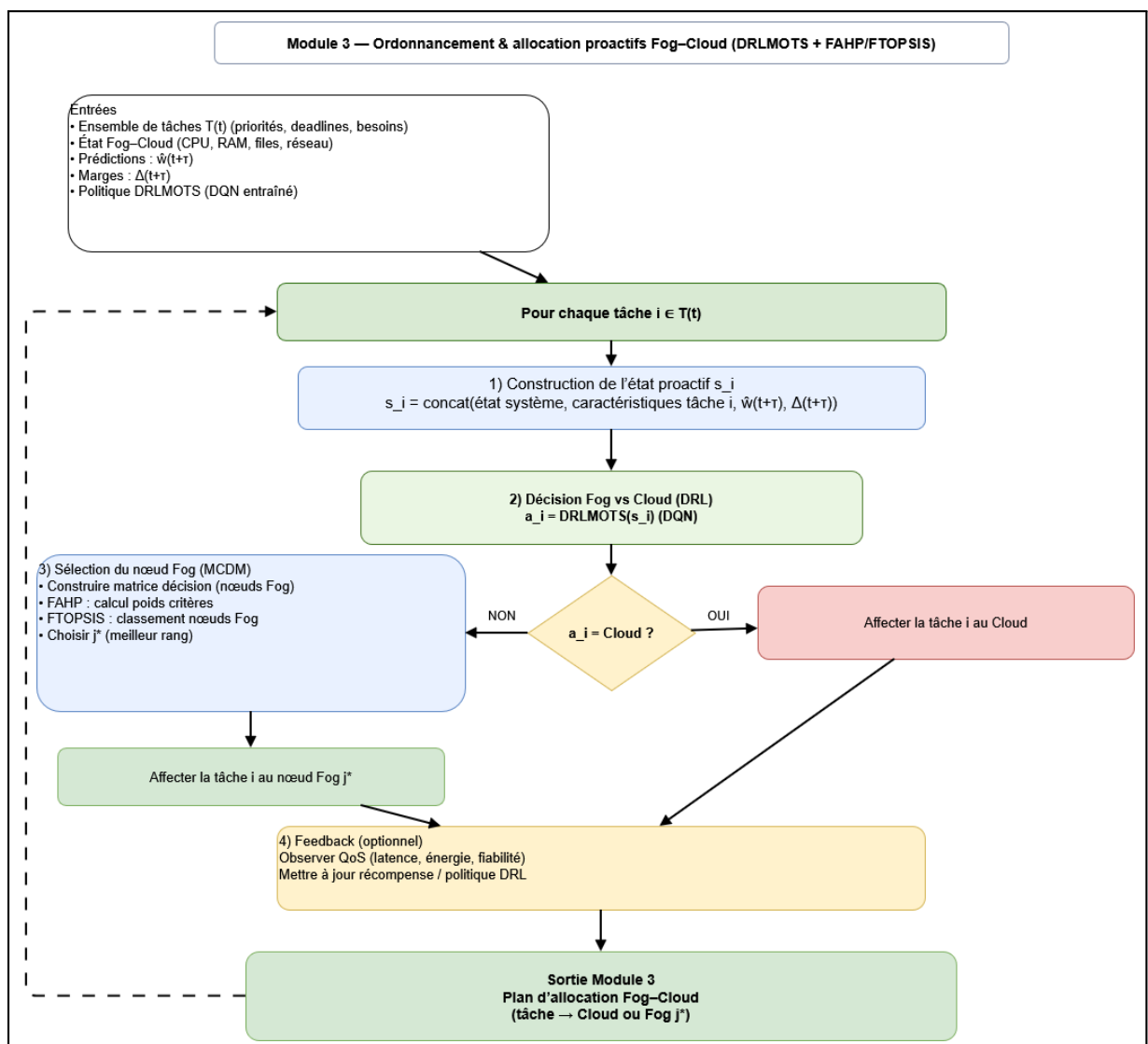


FIGURE 1.4 – Diagramme du Module 3 : ordonnancement proactif Fog–Cloud basé sur DRLMOTS et FAHP/FTOPSIS

Dans un premier temps, nous adoptons **DRLMOTS**, un algorithme d'apprentis-

sage par renforcement profond basé sur un réseau de type DQN, proposé par **R. Choppara** et **K. Mangalampalli** [4]. DRLMOTS est conçu pour la prise de décision séquentielle dans des environnements Fog–Cloud hétérogènes et permet de décider dynamiquement si une tâche doit être exécutée localement au Fog ou *offloadée* vers le Cloud, tout en optimisant simultanément plusieurs objectifs tels que le temps de réponse, la consommation énergétique et la fiabilité.

Dans un second temps, lorsque la décision DRL indique une exécution au niveau Fog, une méthode de sélection multi-critères est appliquée afin d’identifier le nœud Fog le plus adapté. Nous utilisons pour cela une approche combinant **FAHP** et **FTOPSIS**, telle que proposée par Gad-Elrab [6]. Cette méthode permet de classer les nœuds Fog disponibles en fonction de plusieurs critères de QoS, incluant la latence, la charge CPU, la consommation énergétique et la bande passante. L’utilisation conjointe de DRL pour la décision globale Fog/Cloud et de MCDM pour la sélection fine des ressources Fog permet d’éviter toute redondance algorithmique et d’obtenir des décisions à la fois robustes et interprétables.

Afin de rendre l’ordonnancement proactif, la charge prédite $\hat{w}(t + \tau)$ et sa marge d’incertitude $\Delta(t + \tau)$, fournies par le Module 1, sont directement intégrées au processus décisionnel. Ces informations alimentent l’état d’entrée du DQN (DRLMOTS) et interviennent également dans les critères d’évaluation de la méthode FAHP/FTOPSIS. En conséquence, les décisions d’ordonnancement anticipent les risques de congestion et de violation de QoS, plutôt que de se limiter à une réaction fondée uniquement sur l’état instantané du système, en cohérence avec les approches prédictives récentes [5].

L’Algorithme 3 formalise le processus d’ordonnancement proactif Fog–Cloud basé sur DRLMOTS et FAHP/FTOPSIS. Il s’agit d’une intégration directe d’algorithmes existants, sans modification de leur structure interne.

Algorithm 3 Ordonnancement proactif Fog–Cloud basé sur DRLMOTS et FAHP/FTOPSIS

Require: Ensemble de tâches $T(t)$ avec priorités

Require: État courant Fog–Cloud (CPU, RAM, files, réseau)

Require: Prédictions $\hat{w}(t + \tau)$ et marges $\Delta(t + \tau)$

Require: Politique DRLMOTS (DQN entraîné) [4]

Ensure: Plan d'allocation Fog–Cloud

```

1: for chaque tâche  $i \in T(t)$  do
2:   // 1) Construction de l'état proactif
3:    $s_i \leftarrow$  concaténer (état système, caractéristiques de la tâche  $i$ ,  $\hat{w}(t + \tau)$ ,  $\Delta(t + \tau)$ )
4:   // 2) Décision globale Fog vs Cloud (DRL)
5:    $a_i \leftarrow$  DRLMOTS( $s_i$ )
6:   if  $a_i = \text{Cloud}$  then
7:     Affecter la tâche  $i$  au Cloud
8:   else
9:     // 3) Sélection du nœud Fog (MCDM)
10:    Construire la matrice de décision des nœuds Fog
11:    Calculer les poids des critères par FAHP [6]
12:    Classer les nœuds Fog par FTOPSIS [6]
13:    Sélectionner le nœud Fog  $j^*$ 
14:    Affecter la tâche  $i$  au nœud  $j^*$ 
15:   end if
16:   // 4) Feedback (optionnel)
17:   Observer la QoS obtenue (latence, énergie) et mettre à jour la récompense DRL
18: end for
19: return Plan d'allocation Fog–Cloud

```

1.4.6 Environnement de simulation et implémentation

Afin d'évaluer l'architecture proposée dans un contexte Fog–Cloud réaliste, nous utilisons le simulateur **RayCloudSim**, un simulateur **événementiel discret** (*discrete-event*) écrit en **Python**. RayCloudSim s'appuie sur **SimPy** pour la gestion des événements et sur **NetworkX** pour modéliser les topologies réseau. Il permet d'étudier les performances et les coûts des environnements *Cloud/Fog/Edge* et de tester des politiques d'allocation, d'ordonnancement et de routage, tout en restant simple à modifier et à intégrer avec des modules de machine learning en Python.

Les modules décisionnels (prédiction, planning et ordonnancement) sont donc implémentés en **Python**. Plus précisément, le **Module 1** (prédiction multi-horizon basée sur LSTM) et le **Module 3** (ordonnancement DRL de type DQN/DRLMOTS) sont développés avec **PyTorch**, tandis que le **Module 2** (méta-heuristique H-VWPO) et la sélection multi-critères (**FAHP/FTOPSIS**) sont également exécutés en Python.

Comme illustré à la Figure 1.5, l'intégration est désormais **native** : à chaque fenêtre de décision t , RayCloudSim construit l'état du système $\text{state}(t)$ (métriques CPU/-RAM, files d'attente, réseau, tâches arrivées, historique) et appelle directement une fonction de décision (*callback*) $\text{decide}(\text{state})$ du contrôleur Python. Le contrôleur

exécute successivement les Modules 1–3 et renvoie $action(t)$ (décisions de *scaling*, *offloading* ρ et plan d'allocation). Les métriques de QoS observées (latence, coût, énergie, violations SLA) sont ensuite réinjectées dans $state(t+1)$ pour assurer une boucle fermée proactive et continue.

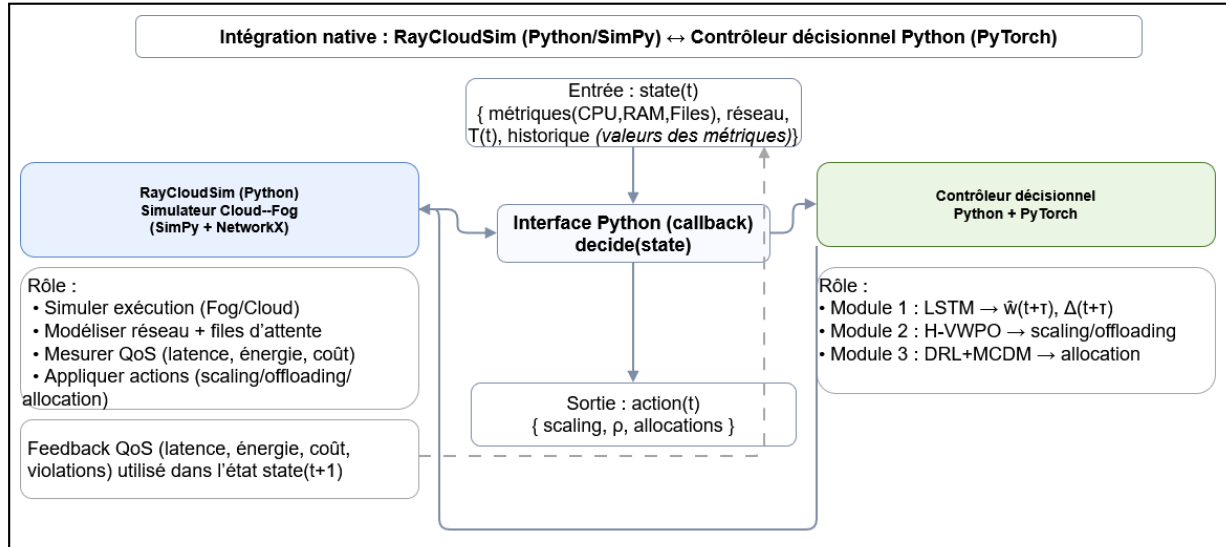


FIGURE 1.5 – Architecture d'intégration RayCloudSim et contrôleur décisionnel en Python pour la simulation Fog-Cloud

- [1] I. Ahmed and Others. Fog computing challenges and opportunities in iot networks : A review. *Journal Name*, 2025.
- [2] O. Akintoye and A. Bagula. Improving quality of service in cloud-fog computing through efficient resource allocation. *Journal Name*, 2019.
- [3] Jing Chen, Yinglong Wang, and Tao Liu. A proactive resource allocation method based on adaptive prediction of resource requests in cloud computing. *Journal of Wireless Communications and Networking*, 2021(24) :1–20, 2021.
- [4] R. Choppara and K. Mangalampalli. An efficient deep reinforcement learning-based task scheduler in cloud-fog environment. *Journal Name*, 2024.
- [5] Thang Le Duc, Chanh Nguyen, and Per-Olov Ostberg. Workload prediction for proactive resource allocation in large-scale cloud–edge applications. *Electronics*, 14(17) :3333, 2025.
- [6] R. Gad-Elrab and Others. Adaptive multi-criteria-based load balancing technique for resource allocation in fog-cloud environments. *Journal Name*, 2024.
- [7] A. Kumar and P. Agrawal. Analysis of multi-dimensional iiot data in edge–fog–cloud architectural frameworks. *Journal Name*, 2023.
- [8] S. Mani and T. Meenakshisundaram. Improving qos in fog computing through efficient resource allocation. *Journal Name*, 2020.
- [9] S. N. Pothu and S. Kailasam. Effective priority-based resource allocation for proactive auto-scaling framework in workload prediction. *Journal Name*, 2024.
- [10] Fatma M. Talaat. Effective prediction and resource allocation method (epram) in fog computing environment for smart healthcare system. *Multimedia Tools and Applications*, 81(6) :8235–8258, 2022.